SPRINT 3 · DEL DATO AL SERVICIO · S03.2

Bases de datos NoSQL

No todo cabe bien en filas y columnas. Una demostración corta de **cuándo** conviene otro modelo, y lo hacemos con la misma pregunta que en SQL.

De filas relacionadas a un documento

En SQL, ver una reserva con su alojamiento y su cliente pide un `JOIN` de tres tablas. En un modelo **documental** se guarda la reserva con esa información **embebida**:

```
{"_id": 5760, "canal": "empresa", "noches": 3, "precio_noche": 89.24,
 "cancelada": 1, "alojamiento": {"nombre": "…", "isla": "Tenerife"}}
```

- **Colección** ≈ tabla, pero **sin esquema fijo**; **base de datos** = conjunto de colecciones.
- En el cuaderno usamos `mongomock`: MongoDB **en memoria**, sin instalar nada.

Cuándo usar cada una

	Relacional (SQL)	Documental (NoSQL)
Estructura	tablas fijas, relaciones	documentos flexibles
Mejor para	reservas, facturas, cifras que **deben cuadrar**	reseñas, navegación, catálogo con atributos distintos
Cambiar el esquema	migración	añadir un campo
Punto débil	rigidez	duplicación: si el alojamiento cambia de nombre, hay que actualizar muchos documentos

No es «uno u otro»: muchos proyectos usan **ambos**.

Consultar con `find`

En SQL	En MongoDB
`WHERE canal = 'directo'`	`{"canal": "directo"}`
`WHERE noches > 5`	`{"noches": {"$gt": 5}}`
`WHERE canal IN (…)`	`{"canal": {"$in": [...]}}`
`UPDATE … SET`	`update_one(filtro, {"$set": {...}})`
`GROUP BY`	`aggregate([{"$match": …}, {"$group": …}])`

```
n = coleccion.count_documents({"cancelada": True})
caras = list(coleccion.find({"alojamiento.isla": "Tenerife", "precio_noche": {"$gt": 150}}))
```

Agregaciones: el GROUP BY de MongoDB

```
pipeline = [
    {"$group": {"_id": "$canal",
                "reservas":   {"$sum": 1},
                "canceladas": {"$sum": {"$cond": ["$cancelada", 1, 0]}}}},
    {"$sort": {"reservas": -1}},
]
tasa_canal = list(coleccion.aggregate(pipeline))
```

- Una **pipeline** es una lista de etapas; cada una transforma el resultado de la anterior.
- `$match` ≈ `WHERE` · `$group` ≈ `GROUP BY` · `$sort`/`$limit` ≈ `ORDER BY`/`LIMIT`.
- Como en SQL: **sin filtro, se afecta a todo**.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

ACTIVIDAD

Actividad — 35 min

NoSQL con documentos (o, si usáis MongoDB Compass, la guía de MongoDB)